بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

In the name of Allah, Most Gracious, Most Merciful

CSE 4107
Structured Programming I

Topic: Formatted Input/Ouput

Asaduzzaman Herok
Lecturer | CSE | IUT
asaduzzaman34@iut-dhaka.edu

The `printf` Function

The `printf` Function:

Follow the white board

- Format string: `printf("Showing value of height = %d", height);`
- General format: `printf(string, expression1, expression2, ...);`
- Conversion / Format specifier: `printf("Showing value of height = %d", height)`
- Uncaught error by compilers:

```
printf("%d %d\n", i);          /* Wrong but no compilation error */
printf("%d \n", i, j, k);      /* Wrong but no compilation error */

int i; float x;
printf("%f %d", i, x);         /* Wrong but no compilation error */
```

Source: C Programming Modern Approach Book

The `printf` Function

Format Specifiers:

Specifier	Used For
%c	a single character
%s	a string
%hi	short (signed)
%hu	short (unsigned)
%Lf	long double
%n	prints nothing
%d	a decimal integer (assumes base 10)
%i	a decimal integer (detects the base automatically)

%o	an octal (base 8) integer
%x	a hexadecimal (base 16) integer
%p	an address (or pointer)
%f	a floating point number for floats
%u	int unsigned decimal
%e	a floating point number in scientific notation
%E	a floating point number in scientific notation
%%	the % symbol

Source: https://www.freecodecamp.org/news/format-specifiers-in-c/

The printf Function

Conversion Specifications:

Follow the white board

- Example: `printf("Float value of with 3 digit after . = %.3f", float_var);`
- General Specification: "%m.pX" or "-m.pX" where m and p are integer constants and X is a specifier letter.

```
printf("%10d %-5d\n", int_var_x,int_var_y);
printf("%10.5f %-5.3f\n", float_var_x, float_var_y);
```

- Minimum field width: m specifies the minimum number of characters to print. +m makes right justified, -m makes left justified.
- Precision: p denotes the precision and depends on X.

Source: C Programming Modern Approach Book

The `printf` Function

Conversion Specifications:

Follow the white board

General Specification: **"%m.pX" or "-m.pX"** where `m` and `p` are integer constants and X is a specifier letter.

Effect of p:

- `%d:` minimum number of digits to display (extra zero added if necessary)
- `%e, %f:` how many digit should appear after decimal point (default is 6)
- `%g:` for floating point number in either exponential or fixed decimal (depends on the size of the number)

Source: C Programming Modern Approach Book

The `printf` Function

Escape sequences:

Name	*Escape Sequence*
Alert (bell)	`\a`
Backspace	`\b`
Form feed	`\f`
New line	`\n`
Carriage return	`\r`
Horizontal tab	`\t`
Vertical tab	`\v`
Backslash	`\\`
Question mark	`\?`
Single quote	`\'`
Double quote	`\"`

Source: C Programming Modern Approach Book

The `scanf` Function

The `scanf` Function:

Follow the white board

- Format string: `scanf("%d%%d%f%f", &i, &j, &x, &y);`
- General format: `scanf(string, &variable1, &variable2, ...);`
- Uncaught error by compilers:

```
scanf("%d %d", i, j);          /* Wrong but no compilation error */
```

Source: C Programming Modern Approach Book

The `scanf` Function

How `scanf` works:

Follow the white board

- Pattern-matching function
- Controlled by format string
- Skips blank space if necessary
- Reads item one by one after each successful read
- Returns immediately if not successful in reading without looking forward
- Ignores **white-space** characters
- Numbers can be put on single line or spread out over several lines.
- It sees one continuous stream of characters
- Peeks at the final new-line without reading it. (will be more clear on the lecture of reading line)

Source: C Programming Modern Approach Book

The scanf Function

Reading numbers:

Follow the white board

- Search er for
  - + or - sign (optional), followed by
  - A series of digits (possibly containing decimal point) followed by
  - An exponent(optional), letter e or E, optional sign, one or more digit
- Encounters a character that cannot be part of the current item, the character is “put back” for the next item or scanf call
  - Input: 1-20.3-4.0e
  - scanf(“%d%d%f%f”, &i, &j, &x, &y)

Source: C Programming Modern Approach Book

The scanf Function

Ordinary Characters in Format String:

Follow the white board

- White-space characters: encounters one or more white-space characters, continues to read until non-white-space is encountered.
  - Space
  - Horizontal and vertical tab,
  - Form-feed: "\f", abbreviated FF, and has ASCII value 12 or 0xC.
  - New-line: ASCII value 10 or 0xA
- Other characters: compares with next input character in the format string. If matches, discards the input character and continues.
  - Doesn’t match? Put back and return
  - Example reading fraction input.

Source: C Programming Modern Approach Book


Q
A


THANK
YOU